STRINGS I

ISTA 130

Defining a String Literal With Quotes: Single Or Double?

- Strings can use either single (') or double (") quotes, but you must match them correctly
  - `'hello'`
  - `'hello'`
  - `'hello"` → `error`
  - `"hello'` → `error`

Various Escape Characters

- '**\n**' is interpreted as a **newline** not an "n"

```
print('x\nx') →
x
x
```

- '**\t**' is interpreted as a **tab**, not a "t"

```
print('x\tx') →
x       x
```

- '\'' is interpreted as a ' not the end of a string!

```
print('x\'x') →
x'x
```

- '\"' is interpreted as a " not the end of a string!

```
print('x\"x') →
x"x
```

Escape Characters

and, if you just want to print a backslash?

- '\\' is interpreted as a plain old \ without the magic.

```
print('x\\x') →
x\x
```

- Think about that for a minute:
  - the normal meaning of a \ in a string is to give special meaning to the next character
  - so a \\ gives special meaning to the second \
  - special meaning for a \ is to just act normal and leave its neighbor alone :)
- Note that an escape character is a *single* character:
  - `print(len('\n'))` → 1
  - `print(len('\t'))` → 1
  - `print(len('\\'))` → 1
  - it just looks like 2 characters to us humans :)

Printing Strings

- What if I don't want spaces between the things I'm printing?

```
name = 'Ziggy'
print('Hello', name, '!')
```

  - prints → `Hello Ziggy !`
  - but I wanted this → `Hello Ziggy!`

We Use String Concatenation

- If we want spaces we need to add them to the string, *inside* the quotes:

```
print('Hello ' + name + '!')
```

- prints this → `Hello Ziggy!`
- But note that this:

```
print('Hello'      + name      +   '!')
```

- still prints → `HelloZiggy!`
- Open a `Python` shell and experiment with concatenation.

Concatenation

- Concatenation works on strings
  - Try this:

```
'hello' + 5
```

```
TypeError:  Can't convert 'int' object to
str implicitly
```

- How can you fix this?
- You already know about the type conversion functions:
  - `'hello' + str(5)` → `hello5`

Type Conversion Functions

When do you need to convert the type of a value?

- We've used the type conversion functions (`int()`, `float()`, `str()`) in several situations
- The value returned by `input()` is a string
- What will this do?

```
age = input('Enter age ')
print('Your age times 10 is', (age * 10))
```

- You have to decide when a type conversion is necessary by looking at the type of value you have and what you need to do with that value:

```
age = input('Enter age ')
print('Your int(age) times 10 is', (int(age) * 10))
```

Concatenation, Repetition

- We've now seen string concatenation:

```
'Hello ' + name
```

- and string repetition:

```
'Hello' * 5
```

- You can of course use them together in more complicated string expressions:

```
('Hello ' + name + '!  ')  * 5
```

- We can do far more interesting things with strings

String Indexing

What will each of these evaluate to?

- `'rabbit'[2]`
- `'elk'[0]`
- `'Cheeze Whiz'[6]`
- `'$1000.00'[0]`
- `'          '[5]`
- `'eggy wegg'[2 * 4]`
- ```
  x = 3
  'lumpy milk'[x]
  ```
- `'123456789'[math.sqrt(100) - 6]`
- `'123456789'[int(math.sqrt(100) - 6)]`
  ```

index	0	1	2	3	4	5	6
character	b	o	l	o	g	n	a

- You can also index into a variable that refers to a string, try this in a shell:

```
word = 'bologna'
word[0]
word[1]
```

- What would this do?

```
word = 'bologna'
word = word + word[0]
```

- Try it in a shell

Looping Through Bologna – Traverse by Index

- We can *traverse* the string (i.e. visit each item in the sequence) using the indices and a `for`-loop

traverse

to move along or over

```
word = 'bologna'
for i in range(len(word)):
    print(word[i])
```

- Or we could use a `while`-loop:

```
word = 'bologna'
i = 0
while i < len(word):
    print(word[i])
    i = i + 1
```

Building Strings

- You can build strings just like you can accumulate a sum:

```
def square_string(n):
    result = ''  # start with empty string
    for i in range(n - 1):
        result += str((i + 1)**2) + ' - '
    result += str(n**2)
    return result
```

Looping Through Bologna – Traverse by Element

There's even a simpler way.

- The `Python` for-loop *iterates* over a *sequence*
  - (we'll talk more about sequences in a few weeks)
- Since a string is a sequence of characters the for-loop can iterate over each character in the sequence:

```
word = 'bologna'
for letter in word:
    print(letter)
# there's nothing special about the name 'letter' here, we
could have used 'toejam' or whatever
```

String `s`: Traverse by Index vs. Traverse by Element

Traverse by Index	Traverse by Element
Loop control variable is assigned `0, 1, 2, …, len(s) - 1`	Loop control variable is assigned `s[0], s[1], …, s[len(s) - 1]`
Loop control variable type is `int`	Loop control variable type is `str`
Can always traverse by index	Won't work if you need to use the index in your loop (because you don't have it – you only have the characters)
Who indexes the string: you do	Who indexes the string: Python does

Negative Indexing

Python also allows us to use *negative* indices.

- negative indices are offsets from the *end* of the string counting backwards
- negative indices allow us to conveniently refer to something at the end of a string
  - `word[-1]` corresponds to `word[len(word) - 1]`
  - `word[-2]` corresponds to `word[len(word) - 2]`

Danger – other languages have serious problems with this idea!

Indexing 'Bologna'

+ index	0	1	2	3	4	5	6
character	b	o	l	o	g	n	a
- index	-7	-6	-5	-4	-3	-2	-1

The `in` Operator

- The **in** operator tests for membership
- It takes 2 operands (e.g. `X in Y`) and evaluates to **True** if the first operand is contained within the second operand. Try each of these in the shell:

```
"a" in "bologna"
```

```
"log" in "bologna"
```

```
"egg" in "bologna"
```

```
"" in "bologna"
```

- The empty string is a member of all strings
- You can use **not in** to see if the first operand isn't inside the second

```
"egg" not in "bologna"
```

String Comparisons

We've seen that strings can be compared using:

```
==
!=
<
<=
>
>=
```

- Try these in a shell:

```
"beer" == "beer"
"apples" < "cheese"
word = "bologna"
word > "ham"
word <= "bologna"
word >= "bologn"
```

String Methods

- A method is a function that is called *on* a particular object
- Recall from turtle graphics:

```
yertle.forward(100)
```

  - This calls the forward method *on* a particular turtle object
  - We use the *dot operator*

```
some_object.some_method()
```

  - You learned many other turtle *methods*
- Contrast this with a call to `print`:

```
print('hello')
```

  - We don't do: `something.print('hello')`

String Methods

There are a number of *methods* that can be applied to strings.

- Try the following example in a shell:

```
'hello'.upper()
```

  - now try this:

```
word = 'hello'
word.upper()
```

- This won't work:

```
upper('hello')
```

  - Since `upper()` is a string method, we must call it *on* a string

String Methods

- To use a string *method* you use *dot notation*:

```
<the_string>.<the_method_name>
```

- For example, to tell the string "BOLOGNA" to call its `lower()` method do this:

```
'BOLOGNA'.lower()
```

- To tell the string "rabbits" to call its `upper()` method do this:

```
'rabbits'.upper()
```

- To have the string that the `name` variable currently refers to call its `upper()` method do this:

```
name = 'Ziggy'
name.upper()
```

Strings Are Immutable

- Since index 0 refers to the first character in the string you might think you can do this:

```
word = "bologna"
word[0] = "B"
```

- `TypeError: 'str' object does not support item assignment`
- Strings are *immutable* which means you can't change them.
- We *can* create *new* strings:

```
word = "bologna"
word = "B" + word[1:]
```

- Here we've created a *new* string by concatenation and we've assigned the new string to the original variable.

String Methods

- Now try this:

```
name = 'Ziggy'
print(name.upper())
print(name)
```

- The `upper` and `lower` methods *do not* modify the original string, they return *new* strings

String Comparisons

- **Beware**: all uppercase characters come *before* all lowercase characters.[1]
- Try these in a shell:

```
"beer" == "Beer"
"beer" < "Beer"
"a" < "Z"
```

- What could you do to make these comparisons come out the way you expect them to?
- Use `str.lower()` (or `str.upper()`) on all strings before comparing them.

[1] Comparisons are based on character encoding scheme (e.g. ASCII). For more see `ord` and `chr` functions at: `http://docs.python.org/3.3/library/functions.html#chr`

+ index	0	1	2	3	4	5	6
character	b	o	l	o	g	n	a
- index	-7	-6	-5	-4	-3	-2	-1

Regular ASCII Chart (character codes 0 - 127)

000		(nul)	016	►	(dle)	032	sp	048	0	064	@	080	P	096	`	112	p
001	☺	(soh)	017	◄	(dc1)	033	!	049	1	065	A	081	Q	097	a	113	q
002	☻	(stx)	018	↕	(dc2)	034	"	050	2	066	B	082	R	098	b	114	r
003	♥	(etx)	019	‼	(dc3)	035	#	051	3	067	C	083	S	099	c	115	s
004	♦	(eot)	020	¶	(dc4)	036	$	052	4	068	D	084	T	100	d	116	t
005	♣	(enq)	021	§	(nak)	037	%	053	5	069	E	085	U	101	e	117	u
006	♠	(ack)	022	▬	(syn)	038	&	054	6	070	F	086	V	102	f	118	v
007	•	(bel)	023	↨	(etb)	039	'	055	7	071	G	087	W	103	g	119	w
008	◘	(bs)	024	↑	(can)	040	(	056	8	072	H	088	X	104	h	120	x
009		(tab)	025	↓	(em)	041	)	057	9	073	I	089	Y	105	i	121	y
010		(lf)	026		(eof)	042	*	058	:	074	J	090	Z	106	j	122	z
011	♂	(vt)	027	←	(esc)	043	+	059	;	075	K	091	[	107	k	123	{
012	♀	(np)	028	∟	(fs)	044	,	060	<	076	L	092	\	108	l	124	|
013		(cr)	029	↔	(gs)	045	-	061	=	077	M	093	]	109	m	125	}
014	♫	(so)	030	▲	(rs)	046	.	062	>	078	N	094	^	110	n	126	~
015	☼	(si)	031	▼	(us)	047	/	063	?	079	O	095	_	111	o	127	⌂

http://vcampus.co/blogs/5190/ascii-table-with-html-entity